

ADAS Minecraft

자율이동 봇 프로젝트

Sensor Fusion | YOLO Object Detection | CNN
Scene Classification
mineflayer-pathfinder + Python Vision Server →
Safe Navigation

Node.js

mineflayer

PyTorch

YOLO

OpenCV

Flask

[목차]

01 서론

- ▶ 자율주행 센서 퓨전 개념
- ▶ mineflayer 동작 원리
- ▶ 프로젝트 아키텍처

02 본론

- ▶ CNN 데이터 수집 & 전처리
- ▶ EfficientNet-B0 구조 & 학습
- ▶ 백본 동결 & 개선점
- ▶ YOLO 데이터 & 라벨링
- ▶ YOLO 학습 & 센서퓨전 연동

03 결론

- ▶ 시연 & 결과 영상
- ▶ 성과 및 한계
- ▶ 향후 발전 방향



김 세 현 : 서론 + CNN (슬라이드 3~10)



장 현 호 : YOLO + 결론 (슬라이드 10~19)

자율주행의 센서 퓨전 (Sensor Fusion)

서론 01

센서 퓨전이란?

여러 센서에서 수집한 데이터를 결합하여
단일 센서보다 정확하고 신뢰성 있는
환경 인식을 달성하는 기법

실제 자율주행 예시

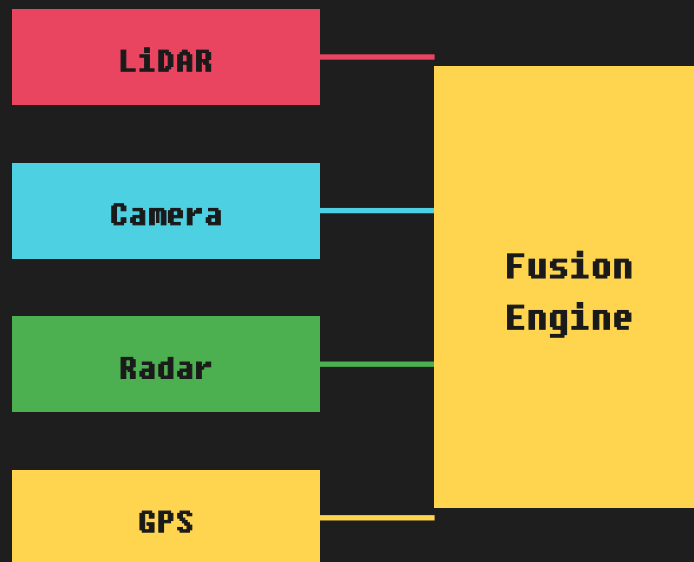
LIDAR → 3D 거리/형태 측정

Camera → 색상/텍스처 인식

Radar → 속도/거리 탐지

GPS → 절대 위치

각 센서의 약점을 다른 센서가 보완



실제 자율주행 ↔ 프로젝트 센서 대응

서론 02

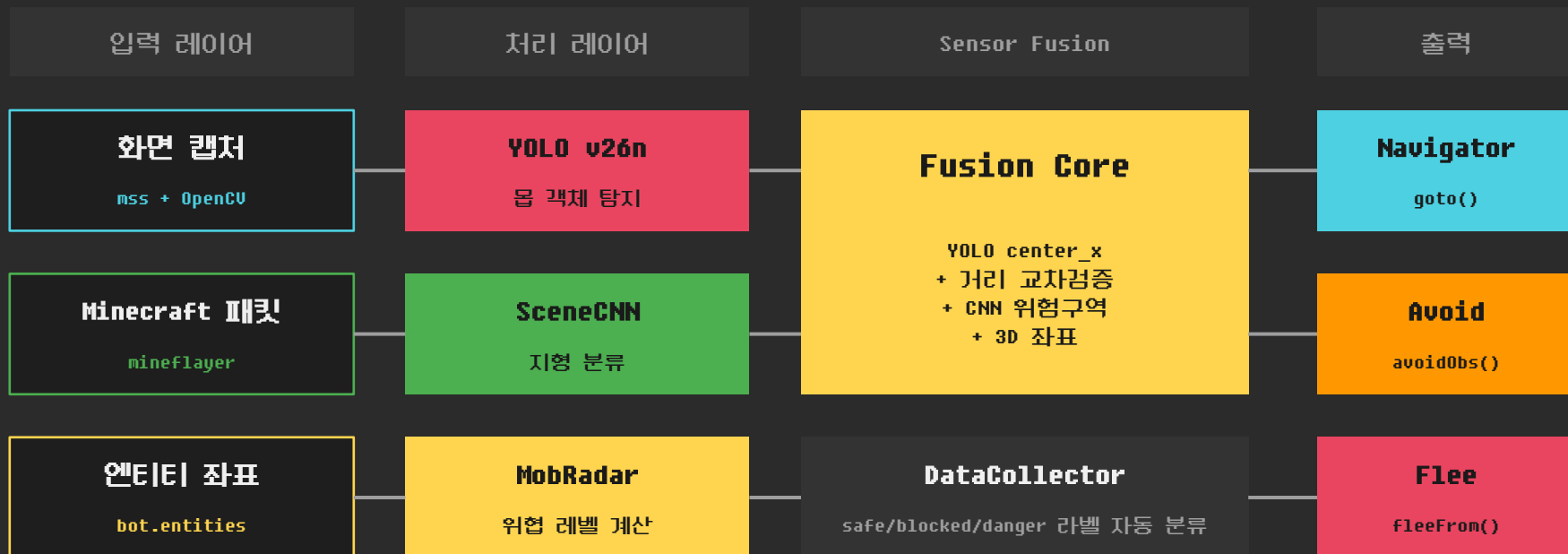
실제 자율주행 차량의 센서 구조를 마인크래프트 환경에서 1:1로 재현

실제 자율주행	=	본 프로젝트 (Minecraft)	대응 역할
Camera (카메라)		화면 캡처 (mss + OpenCV)	색상/텍스처 → 장면 인식
LIDAR (라이다)		Mineflayer bot.blockAt()	3D 블록 좌표 → 지형 스캔
Radar (레이더)		MobRadar (bot.entities)	거리/속도 → 위협 레벨 계산
센서 퓨전 ECU		Fusion Core (index.js)	다중 데이터 → 통합 판단
자율주행 판단		Navigator + CombatManager	회피/도주/정지 → 안전 이동

동일한 원리: 각 센서의 약점을 다른 센서가 보완 → 단일 센서 대비 정확도·신뢰성 향상

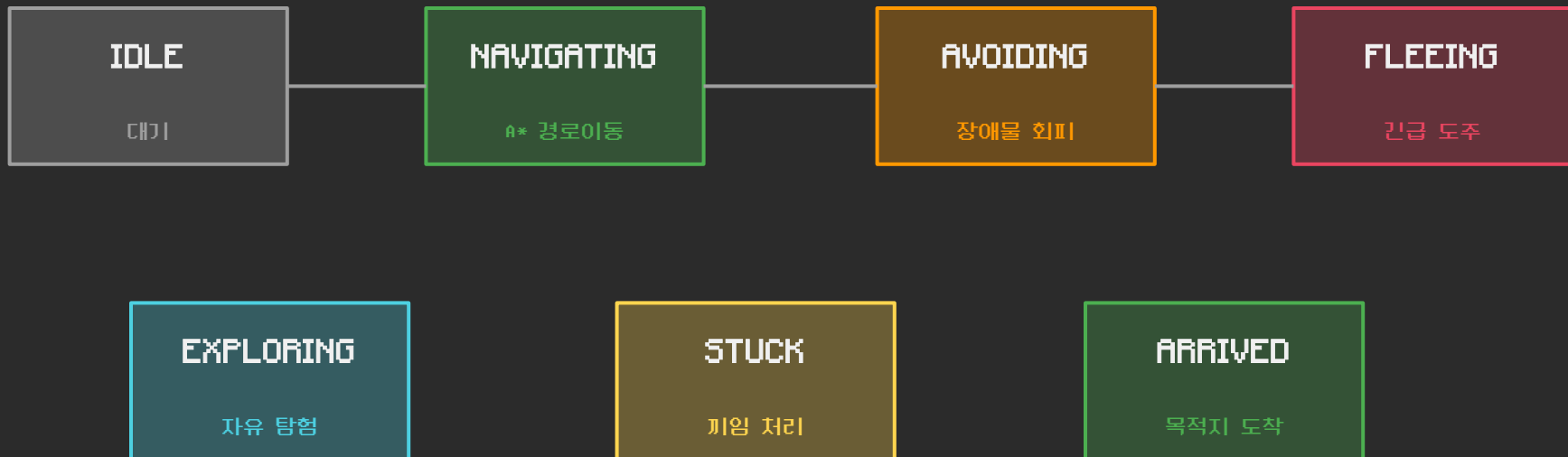
프로젝트 센서 퓨전 아키텍처

서론 03



자율이동 메커니즘 (Navigator)

서론 04



CombatManager: $HP \leq 8$ (4하트) → 긴급 도주 모드 자동 발동 | $HP \geq 14$ 복귀

대시보드: localhost:3000 (SSE 실시간) | 뷰어: prismarine-viewer (1인칭)

CNN 데이터 수집 및 가공

본론 01

1 봇 탐험

!explore 명령으로
봇 자율 탐험

2 화면 캡처

mss로 1인칭
뷰 스크린샷

3 상태 판별

DataCollector가
safe/danger 분류

4 폴더 저장

/collect API로
라벨 폴더에 저장

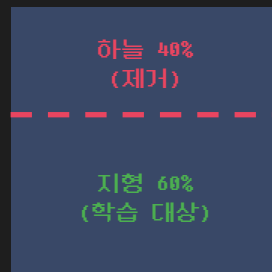
5 전처리

Sky Crop 40%
+ Resize 128x128

데이터셋 폴더 구조

```
dataset/processed_frames/  
├─ 0_safe/  
├─ 1_blocked/  
├─ 2_danger/  
├─ 2_danger_lava/  
├─ 2_danger_cliff/  
└─ 2_danger_water/
```

Sky Crop 전처리 (핵심 설계)



원본 화면에서 상단 40%는
하늘/구름으로 학습에 불필요

하단 60%만 남겨 지형/지면
핵심 정보에 집중

→ 자율주행 도로 인식과 동일 원
리

왜 EfficientNet-B0인가?

EfficientNet-B0 선택 이유



ImageNet 사전학습

120만 장 학습된 특징 추출기
→ 적은 데이터로도 높은 정확도



경량 모델 (5.3M params)

ResNet-50(25M)보다 5배 가벼움
→ 실시간 추론에 유리



Compound Scaling

깊이/너비/해상도를 균형 있게 확장
→ 최적의 효율-정확도 비율



전이 학습(Transfer Learning)

ImageNet 패턴을 마인크래프트에 적용
→ 42,000장으로 94% 달성

모델 구조

Input (3, 128, 128)

RGB 하단 60% 프레임



EfficientNet-B0 백본

MBConv 블록 × 16단계



Global Average Pooling

1280차원 특징 벡터



Dropout (0.3)

과적합 방지



FC → 6 classes

safe / blocked /
danger ×4

백본 동결 & 학습 전략



백본 동결 (Backbone Freeze)이란?

EfficientNet-B0의 특징 추출 레이어(백본)의 가중치를 고정하고 분류 헤드만 학습하는 기법

ImageNet에서 학습된 범용 패턴(에지, 텍스처, 형태)을 보존하면서, 마인크래프트 지형 분류에 특화된 분류기만 새로 학습 → 소량 데이터에서도 안정적 수렴

2-Phase 학습 전략



Phase 1 (Epoch 1~10)

백본 동결, 분류 헤드만 학습 | LR: 1e-3 | Batch: 1024



Phase 2 (Epoch 11~100)

전체 Fine-tuning | LR: 1e-4 | GPU 증강 (RTX 5090)



개선점: 이전 모델 대비 변경 사항

3-Block CNN (수동 설계)

입력 48×48 해상도

Epoch 30, LR 고정

CPU 증강, Batch 32

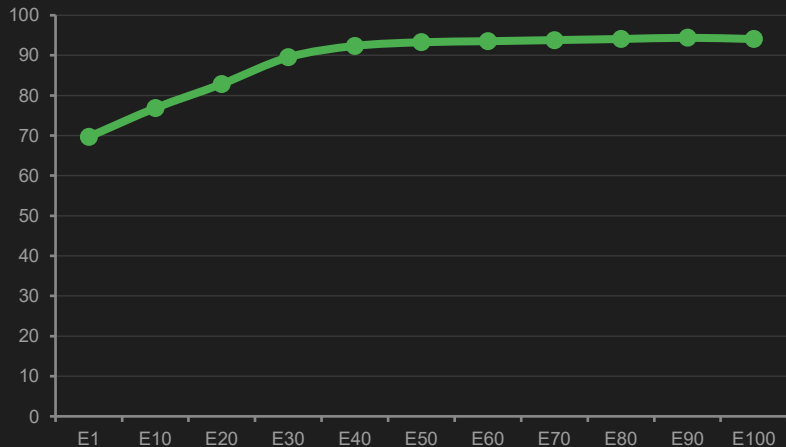
Val Acc ~85% (예상)

- EfficientNet-B0 (사전학습 전이)
- 128×128 해상도 → 세밀한 특징 포착
- 2-Phase (동결→해동), 100 에폭
- GPU 증강 (RTX 5090), Batch 1024
- Val Acc 94.39% (실제 달성)

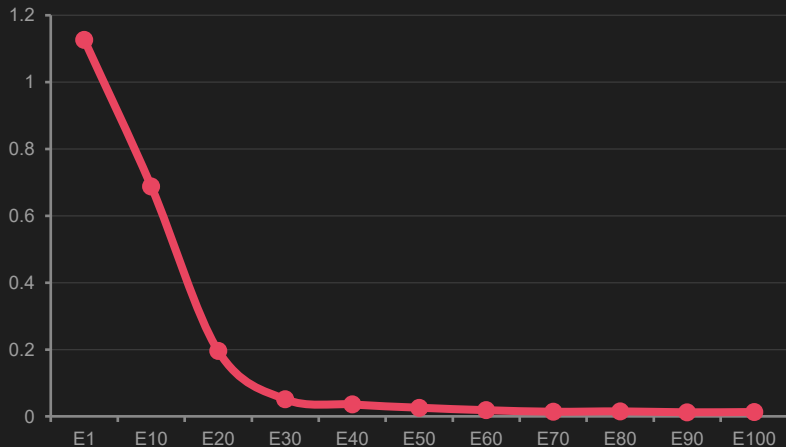
CNN 학습 결과

본문 04

Validation Accuracy (%)



Training Loss



33,942장

학습 데이터

8,486장

검증 데이터

76.86%

Phase 1 최종

94.39%

Phase 2 최고

0.0130

최종 Loss

YOLO 데이터 수집 및 라벨링

본문 05

데이터 수집 방법

1. 마인크래프트 몹 데이터셋 활용
각 몹이 등장하는 장면 스크린샷 데이터
2. 다양한 환경에서 촬영
낮/밤, 실내/야외, 가까이/멀리
여러 각도, 단독/군집 등장
3. 탐지 대상 클래스 (17종)
bee, chicken, cow, creeper, enderman, fox, frog, ghost, goat, llama, pig, sheep, skeleton, spider, turtle, wolf, zombie

라벨링 및 학습 설정

도구	RoboFlow / LabelImg
형식	YOLO TXT 형식
분할	Train 78.2% / Valid 15.4% / Test 6.4%
증강	Flip, Mosaic, MixUp, HSV
입력 크기	640 × 640 px
Conf 임계값	0.30 이상만 인식
데이터 수	17,000개 이미지
모델	YOLOv26n (best.pt)

YOLO 학습 및 센서 퓨전 연동

본문 06

몹 위험도 분류

위험 (DANGEROUS)

creeper, zombie, skeleton, spider, ghast, enderman

중립 (NEUTRAL)

wolf (도발 시 공격)

안전 (SAFE)

pig, cow, sheep, chicken, bee, fox, frog, goat, llama, turtle

센서 퓨전 판단 로직

1

YOLO가 화면 중앙(0.3~0.7)에서
위험 몹 탐지

2

MobRadar로 실제 3D 거리
5블록 이내인지 교차 검증

3

CNN이 전방 지형을
safe/danger 분류

4

BOTH 조건 충족 시
avoidObstacle() 또는 fleeFrom()



YOLO 1차 학습

본문 07

탐지 결과



학습 결과

정밀도  68%

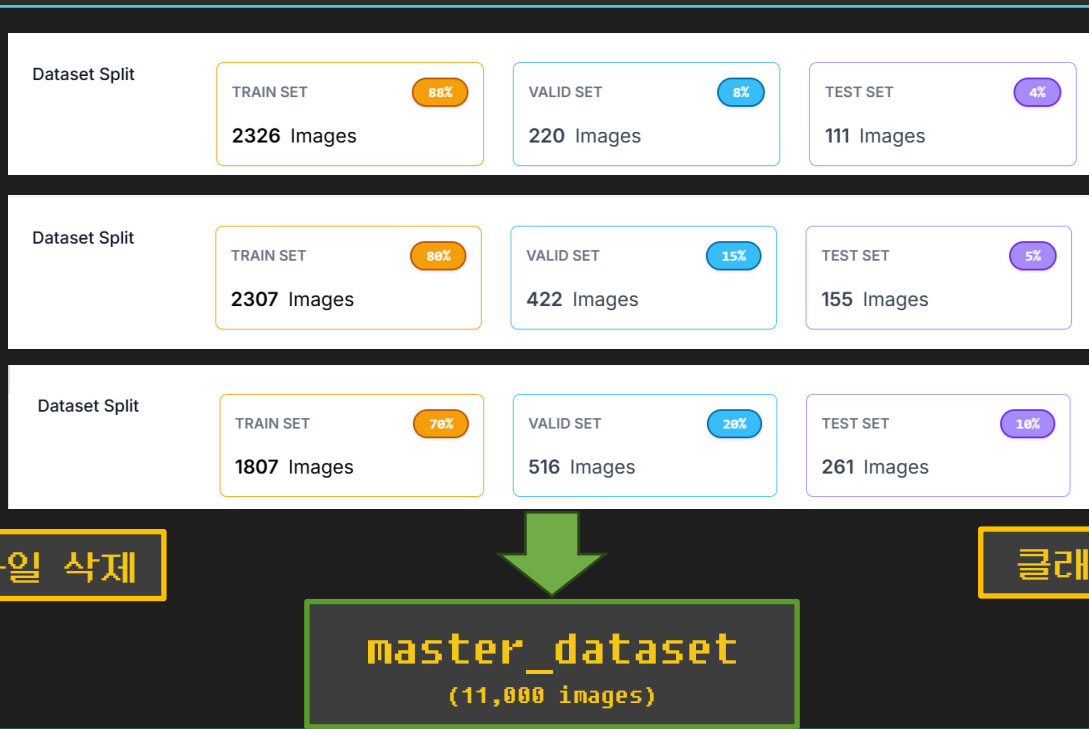
재현율  45%

MAP50  51%

낮은 탐지율, 높은 오탐지율

YOLO DataSet 통일

본론 08



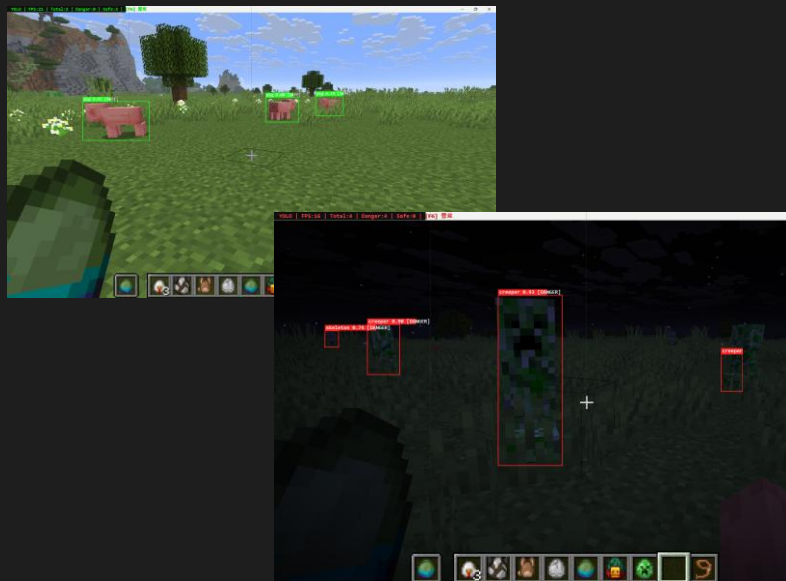
빈 텍스트 파일 삭제

클래스 번호 통일

YOLO 2차 학습

본문 89

탐지 결과



학습 결과

정밀도



81%

재현율



72%

MAP50



76%

높아진 탐지율과 정확도

YOLO 데이터 불균형

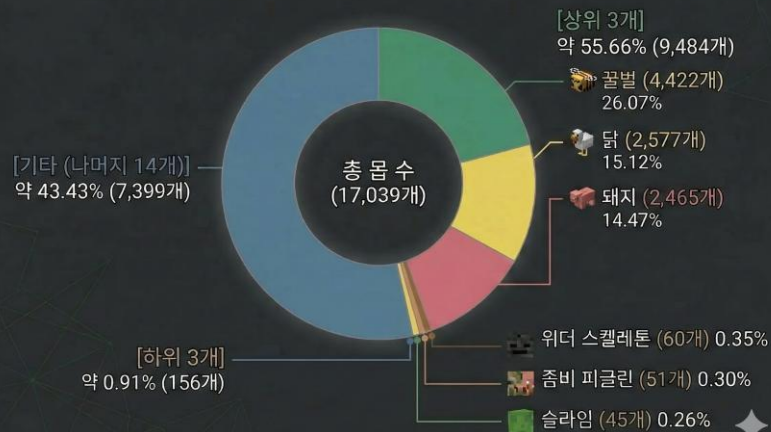
본문 10

일부 몸 낮은 탐지율



데이터 불균형

마인크래프트 몸 비율 분포 분석 (상위 3개, 기타, 하위 3개)



부정확한 데이터셋

라벨링 직접 재확인

거짓 탐지 억제를 위한 배경
데이터 추가

데이터 양이 아닌 라벨 품질
개선을 통해 모델 성능 향상

데이터 불균형

하위 3개 몹을 대량 소환하여
데이터 확보

소수 클래스 데이터 증강

모델 학습 손실 함수 및
가중치 조절

시연 및 결과



시연 시나리오

-  좌표 이동 `!goto x y z` 명령 실행
-  몸 감지 YOLO 탐지창 표시
-  장애물 회피 creeper 우회 경로
-  CNN 분류 지형 위험도 HUD
-  대시보드 실시간 웹 모니터링
-  도착 완료 목적지 안전 도달

성과: YOLO + CNN + mineflayer 3중 퓨전 구조 구현 완료

한계: 야간/동굴 등 저시인성 환경 정확도 저하



감사합니다

ADAS Minecraft 자율이동 봇

Sensor Fusion: mineflayer Entity + YOLO + SceneCNN

Stack: Node.js | Python | PyTorch | YOLOv26n | WebSocket | OpenCV



Q & A